# Լաբորատոր աշխատանք N10

## Աշխատանքի նպատակը

Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ կառավարել դինամիկ հիշողությունը, իրականացնել զանգվածի չափի անվտանգ փոփոխություն (append, removeAt) և կատարել վարկերի ամսական տոկոսների հետ կապված վերլուծական հաշվարկներ C++ լեզվով:

## Տեսական մաս

- **Լռելյայն կոնստրուկտորը** կանչվում է առանց արգումենտների: Դատարկ օբյեկտում n = 0 և_data = nullptr արժեքները նկարագրում են վավեր վիճակ:
- **Դինամիկ հիշողության կառավարում**: Զանգվածի չափը փոխելիս (append կամ removeAt ժամանակ) անհրաժեշտ է նախ հատկացնել նոր հիշողություն (new double[...]), պատճենել հին տվյալները նորի մեջ, ավելացնել կամ հեռացնել համապատասխան տարրը, և միայն դրանից հետո ազատել հին հիշողությունը (delete[])՝ խուսափելու հիշողության արտահոսքերից (memory leaks):
- **Վավերացում**: Բոլոր մուտքային տվյալները և չափերը պետք է ստուգվեն սխալներից խուսափելու համար:

## Խնդրի պահանջներ («Վարկերի ամսական տոկոսներ»)

1. **Դասի կառուցվածք (LoanPortfolio):**
    - private դաշտեր՝ double* data (մայր գումարների զանգված), int n (զանգվածի չափ), double p (ամսական տոկոսադրույք):
    - Լռելյայն կոնստրուկտոր (n = 0, data = nullptr, p = 0):
    - Պարամետրերով կոնստրուկտոր և դեստրուկտոր (~LoanPortfolio()):
    - Մեթոդներ՝ append(value), removeAt(index), print():
2. **Հաշվարկներ**:
    - Մեկ ամսվա տոկոսների ընդհանուր գումար՝ $\sum(debt \cdot p/100)$:
    - Ամենամեծ տոկոսագումարով վարկի ինդեքսը:
    - Մայր գումարների միջին արժեքից փոքր վարկերի քանակը:
3. **Թույլատրելի արժեքներ և սահմանափակումներ**:
    - Բոլոր մայր գումարները պետք է լինեն $> 0$, իսկ $0 \le p \le 100$:
    - Սկզբնական չափը՝ $3 \le n \le 30$:

```
#include <iostream>

#include <iomanip>

class LoanPortfolio {

private:
```

```
    double* data;
    int n;
    double p;
public:
    // 1. Լռելյայն կոնստրուկտոր
    LoanPortfolio() : data(nullptr), n(0), p(0.0) {}
    // Պարամետրերով կոնստրուկտոր
    LoanPortfolio(double initial_p, const double* initial_data = nullptr, int initial_n = 0) {
        p = (initial_p >= 0 && initial_p <= 100) ? initial_p : 0.0;
        n = (initial_n >= 0 && initial_n <= 30) ? initial_n : 0;
        if (n > 0 && initial_data != nullptr) {
            data = new double[n];
            for (int i = 0; i < n; ++i) {
                data[i] = initial_data[i];
            }
        } else {
            data = nullptr;
            n = 0;
        }
    }
    // Դեստրուկտոր
    ~LoanPortfolio() {
        delete[] data;
    }
    // Տվյալների ավելացում (append)
    bool append(double value) {
        if (value <= 0 || n >= 30) return false;
        double* newData = new double[n + 1];
```

```
    for (int i = 0; i < n; ++i) {
        newData[i] = data[i];
    }
    newData[n] = value;
    delete[] data;
    data = newData;
    n++;
    return true;
}
// Տվյալի հեռացում ինդեքսով
bool removeAt(int index) {
    if (index < 0 || index >= n || n <= 0) return false;
    if (n == 1) {
        delete[] data;
        data = nullptr;
        n = 0;
        return true;
    }
    double* newData = new double[n - 1];
    for (int i = 0, j = 0; i < n; ++i) {
        if (i == index) continue;
        newData[j++] = data[i];
    }
    delete[] data;
    data = newData;
    n--;
    return true;
}
```

```
    // Տպելու մեթոդ
    void print() const {
        std::cout << "Տոկոսադրույք (p): " << p << "%, Քանակ (n): " << n << "\nՄայր գումարներ (debt): [";
        for (int i = 0; i < n; ++i) {
            std::cout << data[i] << (i < n - 1 ? ", " : "");
        }
        std::cout << "]\n";
    }
    // Հաշվարկ 1. Մեկ ամսվա տոկոսների ընդհանուր գումարը՝ Σ(debt * p / 100)
    double getTotalInterest() const {
        if (n == 0) return 0.0;
        double sum = 0;
        for (int i = 0; i < n; ++i) {
            sum += data[i] * (p / 100.0);
        }
        return sum;
    }
    // Հաշվարկ 2. Ամենամեծ տոկոսագումարով վարկի ինդեքսը
    int getMaxInterestIndex() const {
        if (n == 0) return -1;
        int maxIdx = 0;
        double maxVal = data[0] * (p / 100.0);
        for (int i = 1; i < n; ++i) {
            double val = data[i] * (p / 100.0);
            if (val > maxVal) {
                maxVal = val;
                maxIdx = i;
```

```
            }
        }
        return maxIdx;
    }
    // Հաշվարկ 3. Մայր գումարների միջինից փոքր վարկերի քանակը
    int getCountBelowAverage() const {
        if (n == 0) return 0;
        double sum = 0;
        for (int i = 0; i < n; ++i) {
            sum += data[i];
        }
        double avg = sum / n;
        int count = 0;
        for (int i = 0; i < n; ++i) {
            if (data[i] < avg) {
                count++;
            }
        }
        return count;
    }
};
int main() {
    std::cout << std::fixed << std::setprecision(2);
    double sample_debt[] = {200000, 500000, 300000, 800000};
    int sample_n = 4;
    double sample_p = 1.5;
    // 1. Ստեղծել դատարկ օբյեկտ (լռելյայն p = 0) և ավելացնել տարրերը append-ով
    LoanPortfolio portfolio1;
```

```
    for (int i = 0; i < sample_n; ++i) {
        portfolio1.append(sample_debt[i]);
    }
    // 2. Առանձին օբյեկտ ստեղծել օրինակի p-ով և տվյալներով
    LoanPortfolio portfolio2(sample_p, sample_debt, sample_n);
    std::cout << "=== ՕԲՅԵԿՏ 1 (Լռելյայն p = 0) ==-\n";
    portfolio1.print();
    std::cout << "Ընդհանուր տոկոսների գումար: " << portfolio1.getTotalInterest() << "\n";
    std::cout << "Ամենամեծ տոկոսով վարկի ինդեքս: " << portfolio1.getMaxInterestIndex() << "\n";
    std::cout << "Միջինից փոքր վարկերի քանակ: " << portfolio1.getCountBelowAverage() << "\n\n";
    std::cout << "=== ՕԲՅԵԿՏ 2 (p = 1.5) ==-\n";
    portfolio2.print();
    std::cout << "Ընդհանուր տոկոսների գումար: " << portfolio2.getTotalInterest() << "\n";
    std::cout << "Ամենամեծ տոկոսով վարկի ինդեքս: " << portfolio2.getMaxInterestIndex() << "\n";
    std::cout << "Միջինից փոքր վարկերի քանակ: " << portfolio2.getCountBelowAverage() << "\n\n";
    // 3. Առաջին օբյեկտից հեռացնել ինդեքս 1-ի տարրը (500000)
    std::cout << "=== ՕԲՅԵԿՏ 1-ից հեռացնում ենք ինդեքս 1-ի տարրը ==-\n";
    portfolio1.removeAt(1);
    portfolio1.print();
    std::cout << "Ընդհանուր տոկոսների գումար: " << portfolio1.getTotalInterest() << "\n";
    std::cout << "Ամենամեծ տոկոսով վարկի ինդեքս: " << portfolio1.getMaxInterestIndex() << "\n";
    std::cout << "Միջինից փոքր վարկերի քանակ: " << portfolio1.getCountBelowAverage() << "\n";
    return 0;
}
```